Information Retrieval

WS 2022 / 2023

Lecture 4, Tuesday November 15th, 2022

(Compression, Codes, Entropy)

Prof. Dr. Hannah Bast
Chair of Algorithms and Data Structures
Department of Computer Science
University of Freiburg

UNI FREIBURG

Overview of this lecture

- Organizational
  - Experiences with ES3 — Efficient List Intersection
- Compression
  - Motivation — save space **and** query time
  - Codes — Elias, Golomb, Variable-Byte
  - Entropy — Shannon's famous theorem
entropy-optimal codes
  - **ES4:** two nice proofs → one half of Shannon's theorem + the optimality of Golomb encoding for gap-encoded lists

Experiences with ES3 1/3

- Summary / excerpts
  - Some of you love math, some of you struggle with it

"I really liked that the sheet was more mathematical, actually I enjoy this more than coding"

"It was a different experience then the last two sheets, I hope to not experience this ever again"

"Need to brush up on my math skills"

"No major issues, only spent some time learning LaTeX"

"I noticed the hint on the exercise sheet to turn the page was not as exciting as usual." *But it was more work than usual!*

No math lectures in the B.Sc. poly curriculum → design error

Experiences with ES3 2/3

- Handwriting

  "Computer scientists are used to run algorithms as time-efficient as possible. Writing is an algorithm."

  "A good handwriting is not required to be a computer scientist. So in short I don't care enough to improve it."

  "Girls get peer-pressured into having to write nicely in school, while boys get shit for having girly handwriting."

  "Computer scientist have bad handwriting because they are not doctors. Doctors have worse handwriting."

  "Dijkstra had a mesmerizing handwriting as far as I know."

  http://www.cs.utexas.edu/users/EWD/ewd10xx/EWD1036.PDF

  https://pure.tue.nl/ws/files/1956738/157204.pdf

Experiences with ES3 3/3

- Help with mathematics
  - For those of you with math phobia and who want help with it, please let us know how you think that we can help you

    See last question on ES4 … you can also answer anonymously
  - My experience: math is easier than you think and a lot of math phobia is about barriers in the head … an example:

    Let's assume you have mastered the four basic arithmetic operations **+ * – /** and parenthesized expressions

    Then a complicated expression is work, but not really hard:

    (16 – 70 / (72 – 7 * (12 – 3) + 1)) / 9 + 9 * (3 + 2) – 12 / 3

    It's just like that for most of the mathematics used in CS

Compression 1/6

Number of websites indexed by Google:

1998:	26,000,000 pages
2000:	1,000,000,000 pages
2004:	4,285,199,774 pages
2005:	8,058,044,651 pages

- Motivation
  - Inverted lists can become very large

    Recall: the length of an inverted list of a word = total number of occurrences of that word in the collection
  - In a web-scale collection, the size can be millions, tens or hundreds of millions, or even billions
  - Example single-keyword queries on Google (as of 14.11.2022)

algorithm	"About 619,000,000 results"
retrieval	"About 193,000,000 results"
and, of, the	"About 25,270,000,000 results"

  - In this lecture, we will learn how **compression** can potentially save not only **space** but **also time**

Compression 2/6

When combined with "skipping" during intersection / merge, one can skip compressed parts

- Index on **a hard disk** (typical for very large data)
  - Compression obviously saves disk space, but also **time** … here is an example with some realistic parameters:

Disk transfer time:	50 MB / second
Compression rate:	Factor 5
Decompression time:	33 MB / second

  - Then fetching an inverted list of size 50 MB takes:

Reading uncompressed:	1.0 seconds	→ 50 MB
Reading compressed:	0.2 seconds	→ 10 MB
Decompressing:	0.3 seconds	→ 50 MB

That is: reading compressed + then decompress is **twice as fast** compared to reading uncompressed

Compression 3/6

- Index on a **SSD** (solid-state drive) or in **RAM**
  - The (realistic) transfer rates are much larger than for a (rotating) hard disk

HDD:	50 – 100	MB / s
SSD:	200 – 500	MB / s
RAM:	4 - 20	GB / s

  - Then it's hard to save not only space but also time by compressing the data

    The decompression speed of **gzip** is around 50 MB / s

  - However: by compressing the data, you might be able to store all your data on SSD or RAM, whereas without compression you might have to resort to a cheaper disk

    1 TB HDD: ~25 €, 1 TB SSD: ~100 €, 1 TB RAM: ~ 5000 €

Compression 4/6

- Gap encoding
  - Example inverted list (doc ids only):

    3, 17, 21, 24, 34, 38, 45, …, 11876, 11899, 11913, …
  - Numbers small in the beginning and large in the end, using an **int** requires **4 bytes** or even **8 bytes** per id
  - Alternative: store differences from one item to next:

    +3, +14, +4, +3, +10, +4, +7, …, +12, +23, +14, …
  - This is called **gap encoding**
  - Works as long as we process the lists from left to right

    Now we have a sequence of mostly (but not always) small numbers … how do we store these in little space?

Compression 5/6

- Binary representation
  - We can write number x in binary using $\lfloor \log_2 x \rfloor + 1$ bits

x	binary	number of bits
1	1	1
2	10	2
3	11	2
4	100	3
5	101	3

$\lfloor \log_2 1 \rfloor + 1 = 1$
$\lfloor \log_2 2 \rfloor + 1 = 2$
$\lfloor \log_2 3 \rfloor + 1 = 2$
$\vdots$

  - This encoding is optimal in a sense … see later slides
  - So why not just (gap-)encode like this and concatenate?

    +3, +14, +4, … → 11, 1110, 100, … → 111110100…

Compression 6/6

- Prefix-free codes, definition
  - Decode bit sequence from the last slide: 111110100

    This could be: +3, +14, +4 → 11, 1110, 100

    Could also be: +7, +6, +4 → 111, 110, 100

    Or: +3, +3, +2, + 4 → 11, 11, 10, 100
  - Problem: we have no way to tell where one code ends and the next code begins

    Equivalently: some codes are prefixes of other codes
  - In a **prefix-free code**, no code is a prefix of another

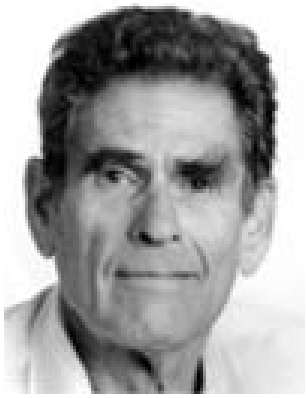
    Then decoding from left to right is unambiguous !

    (It's a good exercise / exam task to prove formally that prefix-freeness and unambiguous left-to-right decoding are equivalent)

Codes 1/4

- Elias-Gamma … from 1975
  - Write $\lfloor \log_2 x \rfloor$ zeros, then x in binary like on slide 9
  - Prefix-free, because the number of initial zeros tells us exactly how many bits of the code come afterwards
  - Code for x has a length of exactly $2 \cdot \lfloor \log_2 x \rfloor + 1$ bits

1	1
2	0 1 0
3	0 1 1
4	0 0 1 0 0
⋮	
10	0 0 0 1 0 1 0



Peter Elias
1923 – 2001

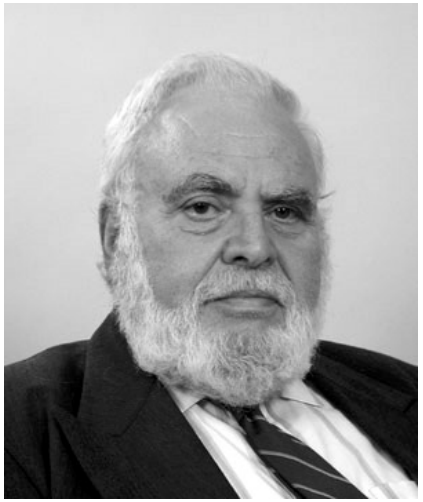
This encoding can be "bootstrapped" (made better by applying it to itself) by replacing the k leading zeros and the following 1 with the Elias-Gamma code for $k+1$. The resulting scheme is called Elias-Delta

Codes 2/4

Code length for code for number x with modulus M is:
$\lfloor x/M \rfloor + 1 + \lceil \log_2 M \rceil$

- Golomb (not Gollum) … from 1966
  - Comes with an integer parameter M, called **modulus**
  - Write x as q · M + r, where q = x div M and r = x mod M
  - The code for x is then the concatenation of:
    - q written in unary with 0s — $\lfloor x/M \rfloor$ bits
    - a single 1 (as a delimiter) — 1 bit
    - r written in binary — $\lceil \log_2 M \rceil$ bits

Solomon Golomb
1932 – 2016



Example: $M = 16$, $x = 42$

$q = \lfloor 42/16 \rfloor = 2$

$r = 42 \% 16 = 10$

0 0 1 1 0 1 0

Codes 3/4

537 in binary: 1000011001

512 16 8 1

25
9
1

512
256

- Variable-Byte (VB)
  - Idea: use **whole bytes**, in order to avoid the (expensive) bit fiddling needed for the previous schemes

    VB is often used in practice, for exactly that reason

    In particular, for UTF-8 encoding … see Lecture 7
  - Use one bit of each byte (for example, the first) to indicate whether this is the last byte in the current code or not

For example: x = 537 = 1000011001 in binary

1	0	0	0	0	1	0	0

0	0	0	1	1	0	0	1

0 = last byte of the sequence

Codes 4/4

- Outlook
  - There is a **huge** variety of codes with different trade-offs

    with respect to: space reduction, compression speed, and decompression speed

  - A beautiful scheme that combines mathematical depth with practical use is

    Asymmetric numeral systems (ANS) ... Duda et al, PCS'14

    It achieves close to optimal space reduction **and** fast (de)compression and is widely used (e.g. Facebook's zstd)

  - The basic idea is to encode a **whole message** (not just individual symbols) into a single (then fairly big) **integer**

    Check out the Wikipedia article if you are interested

Entropy 1/11

- Motivation for coding theory
  - Which code compresses the best ?

    It depends !

    But on what ?
  - Roughly: it depends, on the relative frequency on the numbers / symbols we want to encode

    For example, in natural language, an "e" is much more frequent than a "z"

    So we should encode "e" with less bits than "z"
  - The next slides will make this more precise

Entropy 2/11

$H(X) = -\sum_{i=1}^{m} \frac{1}{m} \cdot \log_2 \frac{1}{m} = -\log_2 \frac{1}{m}$
$= \log_2 m$

- Definition
  - Entropy of a discrete random variable **X**

    Without loss of generality range of X = {1, ..., m}

    Think of X as generating the symbols of a message
  - Then the **entropy of X** is written and defined as

    $H(X) = - \Sigma_i \, p_i \log_2 p_i$ where $p_i = \text{Prob}(X = i)$
  - We will see: H(X) is the optimal number of bits to encode a random symbol generated according to X
  - Example: all $p_i = 1/m$, then $H(X) = \log_2 m$

    If all m symbols are equally likely, the best encoding is the standard encoding with $\log_2 m$ bits / symbol

Entropy 3/11

Where does the + 1 come from in formula (2) below?

This will become clear in the proof

- Shannon's source coding theorem … from 1948
  - Let X be a random variable with finite range
  - For an arbitrary prefix-free (**PF**) encoding, let L_i be the length of the code for $i \in \text{range}(X)$

    (1) For any PF encoding it holds: $\mathbf{E}\, L_X \geq H(X)$

    (2) There is a PF encoding with: $\mathbf{E}\, L_X \leq H(X) + 1$

    where **E** denotes the expectation
  - In words:

    No code can be better than the entropy, and there is always a code that is (almost) as good

Claude Shannon
1916 – 2001

Entropy 4/11

- Central Lemma … to prove the source coding theorem
  - Denote by L_i the length of the code for the i-th symbol, then

    (1) Given a PF code with lengths L_i → $\Sigma_i 2^{-L_i} \leq 1$

    (2) Given L_i with $\Sigma_i 2^{-L_i} \leq 1$ → exists PF code with length L_i

  - Note: $\Sigma_i 2^{-L_i} \leq 1$ is known as "Kraft's inequality"

  - Intuitively: not all L_i can be small … small L_i → large 2^{-L_i}

    For example, the lemma says that a prefix-free code where $L_1 = L_2 = L_3 = 1$ is not possible, because $2^{-1} + 2^{-1} + 2^{-1} > 1$

Entropy 5/11

well-defined procedure when the code is prefix-free

0 1 1 0 0 1

really independence? or just „mutually exclusive"

- Proof of central lemma, part (1)
  - Show: given prefix-free code with lengths L_i then $\Sigma_i 2^{-L_i} \leq 1$

    this means: the whole encoding scheme
  - Generate a random binary sequence as follows:

    1. Pick one bit after the other, and independently
    2. Stop when you have a valid code, or when there is no code left that starts with the bit sequence picked so far
  - Let C_i be the event that the i-th code is generated

    this means: the particular code for symbol i
  - Observe: for a prefix-free code, the C_i are **independent**
  - Thus $Pr(C_1) + \ldots + Pr(C_m) = Pr(C_1 \cup \ldots \cup C_m) \leq 1$
  - We also have $Pr(C_i) = \underbrace{\frac{1}{2} \cdot \frac{1}{2} \cdots \frac{1}{2}}_{L_i \text{ times}} = 2^{-L_i}$
  - This proves $\Sigma_i 2^{-L_i} \leq 1$

Entropy 6/11

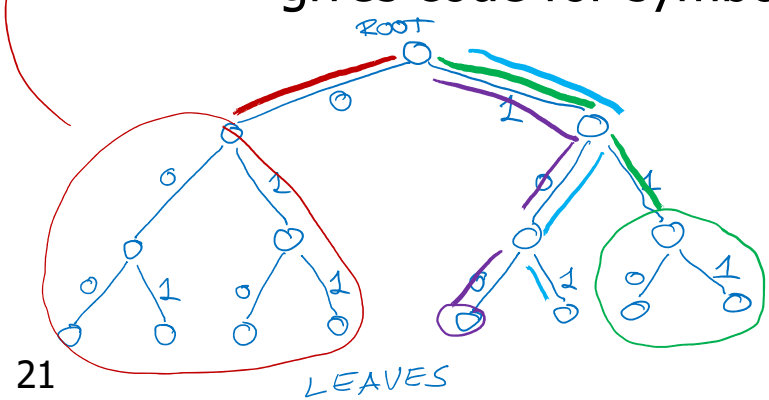
$L_1 = 1$, $L_2 = 2$, $L_3 = 3$, $L_4 = 3$

$\sum_{i=1}^{4} 2^{-L_i} = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{8} = 1$

$M = \max_{i=1..4} L_i = 3$

- Proof of central lemma, part (2)
  - To show: L_i with $\Sigma_i 2^{-L_i} \leq 1$ → exists PF code with lengths L_i
  - Complete binary tree of depth $M = \max L_i$ … has 2^M leaves
  - Mark each left edge 0, and each right edge 1
  - Consider the code lengths L_i in sorted order, smallest first
  - Then iterate: pick subtree with $2^{M - L_i}$ leaves that does not overlap with already picked subtrees … path to that subtree gives code for symbol I … $\Sigma_i 2^{M - L_i} = 2^M \cdot \Sigma_i 2^{-L_i} \leq 2^M$

subtree of size 4



$L_1 = 1$, $2^{3-1} = 4$, code: 0

$L_2 = 2$, $2^{3-2} = 2$, code: 11

$L_3 = 3$, $2^{3-3} = 1$, code: 100

$L_4 = 3$, $2^{3-3} = 1$, code: 101

This is called: HUFFMAN encoding

Entropy 7/11

$\sum p_i = 1$

- Proof of source coding theorem, part (1)
  - We have to show: for any PF encoding $\mathbf{E}\, L_X \geq H(X)$
  - By definition of expectation: $\mathbf{E}\, L_X = \Sigma_i\, p_i \cdot L_i$ (1)
  - By Kraft's inequality: $\Sigma_i\, 2^{-L_i} \leq 1$ (2)
  - Using Lagrange, it can be shown that, under the constraint (2), (1) is **minimized** for $L_i = \log_2 1/p_i$

    This is Exercise 1 from ES4 … it is perfect to practice the Lagrangian multipliers technique once more and for a deeper understanding of the source coding theorem
  - Then $E\, L_X = \Sigma_i\, p_i \cdot L_i \geq \Sigma_i\, p_i \cdot \log_2 1/p_i = H(X)$

Entropy 8/11

now the p_i are given
and we want to find L_i

$\lceil x \rceil \geq x$

- Proof of source coding theorem, part (2)
  - Show: there is a PF encoding with $\mathbf{E}\, L_X \leq H(X) + 1$
  - Let $L_i = \lceil \log_2 1/p_i \rceil$, then $\Sigma_i\, 2^{-L_i} \leq 1$

    $\sum_i 2^{-\lceil \log_2 \frac{1}{p_i} \rceil} \leq \sum_i 2^{-\log_2 \frac{1}{p_i}} = \sum_i 2^{\log_2 p_i} = \sum_i p_i = 1$

    Note that the code length must be an integer, and we must round upwards, so that Kraft's inequality holds
  - By the central lemma, part (2), there then exists a PF code with code lengths L_i
  - By the definition of expectation: $\mathbf{E}\, L_X = \Sigma_i\, p_i \cdot L_i$
  - Hence $\mathbf{E}\, L_X = \sum_i p_i \cdot \underbrace{\lceil \log_2 \frac{1}{p_i} \rceil}_{\leq \log_2 \frac{1}{p_i} + 1} \leq \underbrace{\sum_i p_i \cdot \log_2 \frac{1}{p_i}}_{= H(X)} + \underbrace{\sum p_i}_{= 1}$

Entropy 9/11

- Entropy-optimal codes
  - We say that a prefix-free code with code length L_i for symbol i is **entropy-optimal** for distribution p_i if

    $L_i \leq \log_2 1/p_i + 1$ for all i
  - This implies that

    $\mathbf{E}\, L_X = \Sigma_i\, p_i \cdot L_i \leq \Sigma_i\, p_i \cdot (\log_2 1/p_i + 1) = H(X) + 1$

    By Shannon's theorem this is the best we can hope for

    In words: if a code is entropy-optimal for a distribution, the expected code length is optimal

    (expected = expected with respect to that distribution)

    For Exercise 2 from ES4, to make the task simpler, you only have to prove that $L_i \leq \log_2 1/p_i + \mathbf{C}$ … for some constant C

Entropy 10/11

$2 \log_2 i = \log_2 \frac{1}{p_i}$

$p_i = 1/i^2$

- Entropy-optimality of Elias-Gamma
  - The code length for Elias-Gamma is

    $L_i = 2 \lfloor \log_2 i \rfloor + 1$
  - We want to find a distribution p_i such that

    $2 \lfloor \log_2 i \rfloor + 1 = L_i \leq \log_2 1/p_i + 1$ for all i
  - This suggests $p_i \geq 1 / i^2$ for $i > 1$ because then:

    $2 \lfloor \log_2 i \rfloor \leq 2 \log_2 i = \log_2 i^2 = \log_2 1/p_i$ for $i = 1$: $\log_2 i = 0$
  - We have to take care that the p_i sum to 1:

    $\Sigma_{i=1..\infty} 1 / i^2 = \pi^2 / 6 = 1.6449\ldots$ a fascinating proof by itself !
  - Hence let $p_i = 1 / i^2$ for $i \geq 2$, and $p_1 := 1 - 0.6449$ such that $\Sigma_i p_i = 1$

Entropy 11/11

- Entropy-optimality of Golomb
  - Consider the following random experiment for the generation of an inverted list L of length m :

    Include each document in L with probability $p = m / n$, independently of each other, where n = #documents
  - Let X be a fixed **gap** in this inverted list, then

    $\Pr(X = i) = (1 - p)^{i-1} \cdot p =: p_i$ for $i = 1, 2, 3, \ldots$

    ES4.2: Prove that Golomb encoding with $M = \lceil 1/p \cdot \ln 2 \rceil$ is optimal for this distribution (and hence for gap-encoded lists)

    It's not the most practical scheme though because the encoding involves a lot of bit fiddling and the encoded symbols are not aligned with byte boundaries

References

- Textbook

  Section 5: Index compression

  Section 5.3: Postings file compression (some codes only)

- Wikipedia

  http://en.wikipedia.org/wiki/Elias_gamma_coding

  http://en.wikipedia.org/wiki/Elias_delta_coding

  http://en.wikipedia.org/wiki/Golomb_coding

  http://en.wikipedia.org/wiki/Variable-width_encoding

  http://en.wikipedia.org/wiki/Source_coding_theorem

  http://en.wikipedia.org/wiki/Kraft_inequality